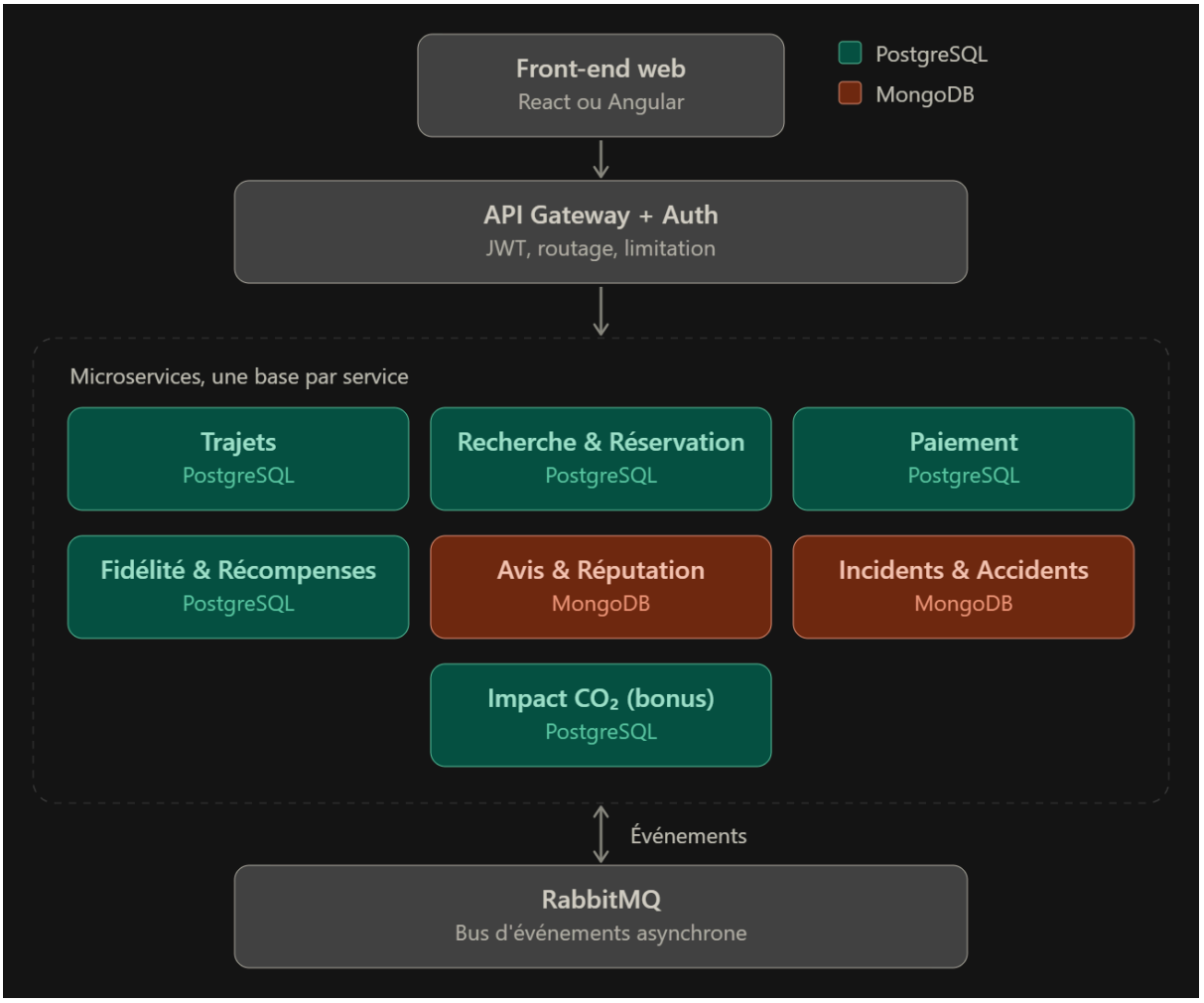
N°	Microservice	Base	Tables	Rôle
1	**Trajets**	PostgreSQL	`rides`, `vehicles`, `ride_stops`, `ride_recurrences`	Créer, modifier, annuler, terminer un trajet
2	**Recherche & Réservation**	PostgreSQL	`ride_index`, `bookings`, `booking_status_history`	Trouver un trajet, demander une place, décompte des places
3	**Paiement**	PostgreSQL	`wallets`, `payments`, `transactions`	Portefeuille simulé, paiement, versement, remboursement
4	**Fidélité & Récompenses**	PostgreSQL	`loyalty_accounts`, `point_transactions`, `badges`, `user_badges`, `rewards`, `redemptions`	Points, badges, niveaux, échange contre du crédit
5	**Avis & Réputation**	MongoDB	`review_rights`, `reviews`, `trust_scores`	Notes mutuelles et score de confiance
6	**Incidents & Accidents**	MongoDB	`incidents`, `incident_confirmations`, `sos_alerts`	Signalements, confirmations, SOS
7	**Impact CO_2** (bonus)	PostgreSQL	`city_distances`, `impact_records`, `impact_totals`	Mesurer le CO_2 économisé et classer les éco-conducteurs



Architecture générale et scénario

Comment lire le schéma

- **Front-end → API Gateway** : le client ne connaît qu'une adresse. La Gateway vérifie le JWT (`userId`, `rôle`), limite les requêtes et route vers le bon service. Chaque service revérifie aussi le JWT lui-même, ce qui explique qu'il n'y ait pas de service Utilisateurs.
- **Microservices** : chacun a sa propre base (PostgreSQL ou MongoDB) et ne lit jamais celle d'un autre.
- **RabbitMQ** : c'est le seul canal entre services. Un service publie un événement, les autres le consomment quand ils veulent, sans se connaître.

Deux types de communication :

- **Synchrone (REST)** : front-end → Gateway → service.
- **Asynchrone (événements)** : service ↔ RabbitMQ ↔ service.

Scénario complet de démonstration

Acteurs : Sami (conducteur), Leila (passagère), un administrateur.

1. Publication du trajet
Sami publie Ariana → Sousse, 3 places, 8 DT. **Trajets** enregistre dans `rides` et publie `RidePublished`.

2. Indexation
Recherche & Réservation reçoit l'événement et ajoute le trajet dans sa copie `ride_index`.

3. Recherche et demande de place
Leila cherche « Ariana → Sousse » et trouve le trajet. Elle demande une place : une ligne `bookings` est créée (REQUESTED) et `BookingRequested` est publié.

4. Acceptation
Sami accepte. Le service décrémente `seats_available`, passe la réservation à CONFIRMED et publie `BookingConfirmed`.

5. Paiement (Saga)
Paiement reçoit `BookingConfirmed`, débite le portefeuille de Leila et bloque le montant (statut HELD). Il publie `PaymentSucceeded`.
Cas d'échec : solde insuffisant, donc `PaymentFailed`. La réservation passe à CANCELLED et la place est libérée automatiquement.

6. Incident pendant le trajet
Sami signale un accident sur l'autoroute. **Incidents** crée un signalement et publie `IncidentReported`. Un autre utilisateur le confirme (`IncidentConfirmed`).

7. Fin du trajet
Sami termine le trajet. **Trajets** publie `RideCompleted`, et quatre services réagissent en parallèle :

- **Paiement** : libère l'argent et crédite Sami.
- **Fidélité** : +10 points à chacun, et un badge si une condition est atteinte.
- **Avis** : autorise la notation entre Sami et Leila.
- **Impact CO_2** : calcule le CO_2 économisé et publie `ImpactUpdated`.

8. Avis
Leila donne 5 étoiles à Sami. **Avis** met à jour son score de confiance et publie `ReviewSubmitted`. **Fidélité** donne 5 points bonus à Leila.

9. Échange de points
Leila échange 50 points contre 2 DT. **Fidélité** publie `RewardRedeemed` et **Paiement** crédite son portefeuille.